

Here's a breakdown of the system architecture for Rootless, followed by a visual overview.

The architecture is organized around four main layers. At the front, a web client handles everything the user interacts with — onboarding inputs (passport country, destination, relocation goal), an AI-powered visa recommendation engine, a personalized checklist and timeline view, and a user dashboard for tracking progress.

Behind that sits an API gateway that authenticates requests and routes traffic to specialized backend services. The core logic lives in three services: a Visa Intelligence Service that queries a structured visa rule database and runs matching logic; a Personalization Engine that uses an LLM (via the Anthropic API) to generate customized checklists and timelines based on the user's situation; and a Content & Compliance Service that manages document templates and deadline logic.

The data layer consists of a user database (profiles and progress), a visa rules database (requirements by passport/destination pair), and a document store (templates and generated checklists). Finally, external integrations pull in government immigration APIs and embassy data feeds to keep visa rules current. Every box in the diagram is clickable to explore a specific component in depth. Here's how the key layers connect end-to-end:

User input → Visa matching. When a user submits their passport country, destination, and goal, the API gateway routes the request to the Visa Intelligence Service. This service queries the Visa Rules DB — a structured store of requirements indexed by passport/destination pairs — and returns a ranked list of eligible visa types with their requirements.

Visa match → Personalized plan. The matched visa type, along with the user's relocation goal and timeline, gets passed to the Personalization Engine. This service calls the Anthropic API with a structured prompt to generate a customized checklist and phased timeline — accounting for pre-departure tasks (document gathering, applications) and post-arrival tasks (registration, bank accounts, health insurance).

Progress and notifications. A Progress Tracker service stores completed checklist items per user in the User DB and feeds the dashboard. A Notification Service handles deadline reminders and status updates (e.g., "your visa appointment is in 7 days").

Keeping data current. The Content & Compliance Service pulls regularly from government immigration APIs and embassy feeds to refresh the Visa Rules DB. This is the most operationally critical piece — visa rules change frequently, and stale data directly harms users.

A few architectural decisions worth calling out as you build this:

The **visa rules database** should be treated as a curated, versioned dataset rather than relying on live government API calls at query time. Government portals are unreliable and inconsistently structured — scraping or ingesting them on a schedule (daily/weekly) and storing a clean internal copy is far more robust.

The **LLM layer** is best used for *generation*, not *lookup*. Visa eligibility rules should be deterministic logic from the database. The LLM's job is turning those rules and the user's context into readable, personalized language — not making legal or procedural determinations on its own.

Auth and multi-tenancy should be designed in from day one. Users will return to their checklist over weeks or months, so session persistence, secure credential storage, and eventually OAuth (for Google/Apple sign-in) are essential early decisions.

